Lab 4 - Polimorfismo

Objetivo

Diseñar e implementar en Java una solución con interfaces, una clase abstracta y subclases, de modo que la tabla de posiciones y el calendario funcionen sin conocer el deporte de cada encuentro ni el tipo de cada evento.

El caso

ArenaGT organiza una copa deportiva entre empresas. En la copa, equipos de distintas empresas juegan encuentros de **fútbol sala** y de **baloncesto**, y todos los resultados suman en una sola tabla de posiciones. Además de los encuentros, la copa incluye **exhibiciones**, como clínicas o demostraciones, que se muestran en el calendario pero no cuentan para la tabla. Hasta ahora llevaban todo en una hoja de cálculo, pero se equivocan seguido al sumar puntos y al armar el calendario, así que quieren un programa que lo haga por ellos.

Equipos. Cada equipo tiene un código que lo identifica, un nombre y la empresa a la que representa. No pueden existir dos equipos con el mismo código. Cada equipo acumula *puntos de tabla*, que se calculan según las reglas de cada deporte. No deben confundirse con los goles o puntos anotados dentro de un encuentro.

Deportes. Ambos deportes se juegan entre dos equipos, y el marcador son dos números enteros no negativos, uno por equipo. Se diferencian en sus reglas:

- **Fútbol sala.** El marcador son los goles de cada equipo y se permiten empates. La victoria otorga 3 puntos de tabla, el empate 1 a cada equipo y la derrota 0.
- **Baloncesto.** El marcador son los puntos anotados por cada equipo y no se permiten empates: un marcador igual se rechaza. La victoria otorga 2 puntos de tabla y la derrota 1.

Por ejemplo, si el equipo Faro vence 3–1 a Sol en fútbol sala, Faro suma 3 puntos de tabla y Sol 0. Si después Sol vence 78–74 a Faro en baloncesto, Sol suma 2 y Faro 1.

Exhibiciones. Una exhibición no es un encuentro: no tiene equipos, marcador ni resultado. Solo tiene un título (por ejemplo, "Clínica de baloncesto") y se programa igual que un encuentro, con fecha, hora y sede. Tiene dos estados: *sin programar* y *programada*.

Encuentros y calendario. El personal crea un encuentro eligiendo el deporte y los dos equipos que jugarán; un equipo no puede jugar contra sí mismo. Un encuentro nace *sin programar*. Luego el personal le asigna una fecha, una hora de inicio (hora completa, de 0 a 23) y una sede, y pasa a *programado*. Puede reprogramarse mientras no tenga resultado, y lo mismo aplica a las exhibiciones. Un evento, sea encuentro o exhibición, no puede programarse si su sede ya tiene otro evento en esa misma fecha y hora. Además, un encuentro no puede programarse si alguno de sus dos equipos ya tiene otro encuentro en esa misma fecha y hora.

El calendario muestra los eventos que ya tienen fecha, encuentros y exhibiciones mezclados, en orden cronológico: por fecha, luego por hora y luego por nombre de sede. Por ejemplo, un evento

el 12 de noviembre a las 9:00 va antes que uno el mismo día a las 14:00, y si empezaran a la misma hora se ordenan por el nombre de la sede. Quien recorra el calendario no debe necesitar saber cómo se guardan los eventos ni repetir el criterio de orden.

Resultados y tabla de posiciones. Solo se puede registrar el resultado de un encuentro programado. Un resultado válido deja el encuentro *finalizado* y suma los puntos de tabla a los dos equipos. Un encuentro finalizado no admite otro resultado. La tabla de posiciones ordena a los equipos por puntos de tabla, de mayor a menor. Si empatan, se ordenan por nombre, en orden alfabético: por ejemplo, con 4 puntos cada uno, Faro va antes que Luna.

Crecimiento. ArenaGT planea agregar más deportes, como voleibol o tenis de mesa, cada uno con su propia forma de validar el marcador y de repartir los puntos, y más tipos de evento. Agregar un deporte no debería obligar a modificar la tabla de posiciones ni el calendario.

Pautas de diseño. Su solución debe tomar estas decisiones:

- **Interfaz para el calendario.** Todo lo que pueda programarse y mostrarse en el calendario debe cumplir una misma interfaz. La cumplen los encuentros y las exhibiciones, que no se parecen entre sí y no deben heredar uno del otro.
- **Interfaz para la tabla de posiciones.** Todo lo que pueda recibir un resultado y repartir puntos de tabla debe cumplir una segunda interfaz. La cumplen los encuentros, pero no las exhibiciones. El motivo de que sean dos interfaces y no una es parte del análisis.
- **Clase abstracta para los encuentros.** Lo que comparten todos los encuentros (los dos equipos, el marcador, el estado y la programación) debe vivir en una clase abstracta que no se pueda instanciar. Esa clase debe dejar sin resolver lo que cambia según el deporte.
- **Una subclase concreta por deporte.** Las reglas de cada deporte (qué marcador es válido y cuántos puntos se reparten) se resuelven en su propia subclase de la clase abstracta.
- **Exhibición como clase concreta.** Cumple únicamente la interfaz del calendario.
- **Equipo y copa como clases concretas.** Los equipos deben poder compararse entre sí para ordenar la tabla. La copa guarda los equipos y los eventos, y debe poder recorrerse por su calendario.
- **Un solo criterio de orden.** El orden cronológico debe escribirse una sola vez y servir tanto para encuentros como para exhibiciones.

La tabla de posiciones y el calendario no deben preguntar por el tipo del objeto para decidir qué hacer: deben apoyarse en las interfaces y en las clases abstractas.

Trabajo solicitado

Análisis

1. Redacte los requisitos funcionales: la información que el sistema debe conservar, las operaciones que debe permitir y las condiciones para que cada operación sea válida.
2. Indique qué reglas dependen del deporte y qué comportamiento comparten los encuentros y las exhibiciones. Para cada uno, diga si lo resuelve una interfaz, una clase abstracta o una clase concreta, cuál, y por qué.
3. Describa los estados por los que pasan un encuentro y una exhibición, y qué operación provoca cada cambio. Puede usar una tabla o un diagrama de estados.
4. Proponga las clases, la clase abstracta y las dos interfaces con sus responsabilidades. Para cada atributo y método, indique su propósito, tipo y visibilidad. Explique cómo reciben sus valores iniciales los objetos, incluyendo cómo colaboran los constructores dentro de la jerarquía.
5. Justifique por qué el calendario y la tabla de posiciones se apoyan en dos interfaces distintas en lugar de una sola, y qué pasaría si las exhibiciones tuvieran que cumplir la interfaz de la tabla.
6. Explique qué tendría que crear o modificar para agregar un tercer deporte, y si la tabla de posiciones y el calendario tendrían que cambiar.

Diseño

Elabore un diagrama de clases UML consistente con el análisis. Debe incluir:

- Atributos, métodos, constructores, visibilidad y relaciones con su multiplicidad.
- Las dos interfaces y la clase abstracta, con la notación que las distingue, y las relaciones de herencia y de implementación diferenciadas entre sí.
- Los métodos abstractos y los que se sobrescriben o implementan, identificables en el diagrama.
- Las clases cuyos objetos pueden compararse entre sí y la que puede recorrerse, con el contrato que cumplen.
- La clase que contiene el método `main`.

Use una anotación para explicar las decisiones que no sean evidentes, como dónde se calculan los puntos de tabla de cada deporte o dónde se escribe el criterio de orden del calendario.

Programa

Implemente la solución como una aplicación de consola en **Java**. El menú debe permitir lo siguiente y permanecer disponible hasta que el usuario decida salir:

- Registrar equipos.
- Crear encuentros y exhibiciones.
- Programar y reprogramar eventos, y registrar resultados de encuentros.
- Mostrar la tabla de posiciones y el calendario.

El programa debe iniciar con datos cargados directamente en el código, sin pasar por el menú:

- Al menos cuatro equipos, de al menos dos empresas.
- Al menos cuatro encuentros, con al menos uno de cada deporte, de los cuales al menos dos ya tienen resultado.
- Al menos una exhibición ya programada.

Elija datos que permitan observar las reglas de puntos de ambos deportes, incluyendo un empate en fútbol sala y un desempate por nombre en la tabla.

La aplicación debe rechazar con un mensaje claro, y sin modificar el estado previo, cualquier operación inválida. Esto incluye:

- Códigos vacíos o repetidos, valores negativos o que no sean números, y horas fuera del rango de 0 a 23.
- Un encuentro de un equipo contra sí mismo.
- Un marcador de baloncesto empatado.
- Programaciones en las que la sede ya está ocupada a esa fecha y hora, o en las que un equipo ya tiene otro encuentro a esa fecha y hora.
- Resultados sobre encuentros sin programar o ya finalizados.

Las entradas con formato incorrecto no deben terminar el programa. Los datos solo se conservan en memoria; no se requieren archivos, bases de datos ni interfaz gráfica.

Pruebas

Documente casos de prueba con datos de entrada, resultado esperado y resultado obtenido. Deben cubrir:

- Los puntos de tabla de cada deporte, incluyendo el empate en fútbol sala.
- El desempate por nombre en la tabla de posiciones.
- El orden del calendario con encuentros y exhibiciones mezclados, y la reprogramación de un evento.

- Los conflictos de sede y de equipo al programar y las demás operaciones rechazadas, con evidencia de que no modifican la tabla de posiciones.

Material a entregar en Canvas

- Archivo PDF con el análisis, el diagrama UML y la evidencia de las pruebas realizadas. Incluya el diagrama dentro del documento; no entregue únicamente un enlace.
- URL de un repositorio público en GitHub con todos los archivos `.java` y un archivo `README.md` con las instrucciones necesarias para compilar y ejecutar el programa.

Evaluación

Componente	Criterio	Puntos
Análisis	Requisitos, reglas de puntuación, estados y condiciones de validez completos y precisos.	10
	Interfaces, clase abstracta y clases concretas con responsabilidades claras. Cada regla que depende del deporte o del tipo de evento está asignada a la abstracción adecuada.	10
	Justificación de las abstracciones elegidas, incluyendo por qué son dos interfaces, y análisis del impacto de agregar un deporte nuevo.	10
	Subtotal	**30**
Diseño	Notación UML correcta, con interfaces, clase abstracta, herencia e implementación distinguibles, y diagrama legible.	8
	Atributos, métodos, constructores, abstracción, sobrescritura, implementación y relaciones representados de forma completa y consistente con el análisis.	12
	Responsabilidades bien distribuidas, estado protegido y sin duplicación de datos ni de reglas entre deportes.	10
	Subtotal	**30**
Programa	Implementación consistente con el diseño. Las reglas de cada deporte y de cada tipo de evento se resuelven mediante polimorfismo y no con verificaciones del tipo del objeto.	10
	Puntos de tabla y tabla de posiciones correctos, incluyendo el desempate.	10
	Calendario ordenado y recorrible mediante Comparable e Iterable, con el criterio de orden escrito una sola vez y sin exponer la estructura interna de las clases.	10
	Menú funcional, conflictos de programación y entradas inválidas manejados sin terminar inesperadamente.	5
	Pruebas con resultados esperados y obtenidos que cubren las reglas y las operaciones rechazadas.	5
	Subtotal	**40**
	Total	**100**